

学号：22920202200773

姓名：孟媛

实验 5 雅可比迭代法求解线性方程组

一、实验目的

使用二分法求方程的根。

二、数值计算原理

雅可比迭代法对方程组进行简单的变形，通过如下迭代公式进行迭代：

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(- \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)} + b_i \right)$$

化为矩阵形式：

对线性方程组 $Ax = b$ ，设系数矩阵 A 非奇异，且 $a_{ii} \neq 0 (i=1, 2, \Lambda, n)$ ，可将 A 分解为

$$A = D - L - U$$

其中

$$D = \begin{pmatrix} a_{11} & & & & \\ & a_{22} & & & \\ & & O & & \\ & & & & \\ & & & & a_{nn} \end{pmatrix}$$
$$-L = \begin{pmatrix} 0 & & & & \\ a_{21} & 0 & & & \\ a_{31} & a_{32} & 0 & & \\ M & M & O & O & \\ a_{n1} & a_{n2} & \Lambda & a_{nn-1} & 0 \end{pmatrix}$$
$$-U = \begin{pmatrix} 0 & a_{12} & a_{13} & \Lambda & a_{1n} \\ & 0 & a_{23} & \Lambda & a_{2n} \\ & & 0 & O & M \\ & & & O & a_{n-1n} \\ & & & & 0 \end{pmatrix}$$

则方程 $Ax = b$ 变为 $(D - L - U)x = b$

化为迭代格式：

$x = (I - D^{-1}A)x + D^{-1}b = B_Jx + f_J$ ，即可使用此格式进行迭代操作。

三、源程序介绍

函数接受四个参数：方程组 $Ax=b$ 的 A 矩阵和 b 矩阵，精度和最大迭代次数，返回迭代路径和最终结果，并输出过程信息。

函数计算过程：

- (1) 初始化精度（最小值 $1e-6$ ）、最大迭代次数（100）和迭代初始列向量 x_0
- (2) 得到对角矩阵 D ，上三角矩阵 U 和下三角矩阵 L
- (3) 当迭代次数超过最大迭代次数或者后一次的解和前一次的解的差的范数达到精度，则停止迭代，输出信息
- (4) 利用得到的新解 y ，使用雅可比迭代法更新 x_0
- (5) 转到 (3)

```
1  # 雅可比迭代法求解线性方程组
2  import numpy
3  def Jacobi(A, b, x, e, times=100):
4      x0 = numpy.linalg.solve(A, b)
5      length, width = numpy.shape(A)
6      D = numpy.mat(numpy.diag(numpy.diag(A)))
7      L = -1*numpy.triu(A, 1)
8      U = -1*numpy.tril(A, -1)
9      H = numpy.eye(length)-D.I*A
10     eig, _ = numpy.linalg.eig(H)
11     spectral_radius = max(abs(eig))
12     if spectral_radius < 1:
13         print('此方程组收敛,谱半径为', round(spectral_radius, 5))
14         k = 0
15         while k < times:
16             if max(abs(x-x0).A1) > e:
17                 x = H*x + D.I*b
18                 k += 1
19             else:
20                 print('当精度为', e, '时,Jacobi在%d次内收敛' % k)
21                 break
22         print('自编函数Jacobi迭代解为:\n', x)
23     else:
24         print('Jacobi迭代法不收敛,谱半径为', round(spectral_radius, 5))
25
26
27     A = numpy.mat([[10, -1, -2], [-1, 10, -2], [-1, -1, 5]])
28     b = numpy.mat('72;83;42')
29     x = numpy.mat('0;0;0')
30     e = 1e-6
31     times = 100
32     Jacobi(A, b, x, e, times)
33     print("自带函数求得的解集为:\n", numpy.linalg.solve(A, b))
```

四、程序执行情况

4.1 书上给的例子：

```
A = numpy.mat([[10, -1, -2], [-1, 10, -2], [-1, -1, 5]])  
b = numpy.mat('72;83;42')
```

```
此方程组收敛,谱半径为 0.33723  
当精度为 1e-06 时, Jacobi在16次内收敛  
自编函数Jacobi迭代解为:  
[[10.99999968]  
 [11.99999968]  
 [12.99999963]]  
自带函数求得的解集为:  
[[11.]  
 [12.]  
 [13.]]
```

4.2 书上未给的例子

```
A = numpy.mat([[-10, 1, -2], [1, 10, -2], [-1, 1, 5]])  
b = numpy.mat('72;-83;42')
```

```
此方程组收敛,谱半径为 0.3  
当精度为 1e-06 时, Jacobi在15次内收敛  
自编函数Jacobi迭代解为:  
[[-9.32293588]  
 [-5.82752282]  
 [ 7.7009174 ]]  
自带函数求得的解集为:  
[[-9.32293578]  
 [-5.82752294]  
 [ 7.70091743]]
```

五、结果分析

分析发现，雅可比迭代法在收敛的情况下可以求得精度很高的解，但因为不能保证其是收敛的所以使用雅可比迭代法并不能保证能找到解。

它的思想与计算比较简单，在收敛的情况下性能优秀，准确度也较为优良。但是如何把握是否收敛，是使用此方法的关键。

六、实验体会

雅克比迭代思想较为简单，代码编写也较为简单，但是，不一定能找到解，我认识到了

它的局限性。同时对雅可比迭代法的实现机理及收敛条件的理解更为加深。